



طراحی سامانه‌های میکروسرویس

نیم‌سال دوم ۱۴۰۴-۰۵

استاد: دکتر علی ابریشمی - سیدحسین میرحسینی

تمرین چهارم

پاسخ‌دهنده: معین آعلی - ۴۰۱۱۰۵۵۶۱

فهرست مسائل

۱		مسئله ۱
۱		الف
۱		ب
۱		ج
۲		مسئله ۲
۳		مسئله ۳
۴		مسئله ۴
۶		مسئله ۵
۸		مسئله ۶

پاسخ مسئله‌ی ۱.

الف

Domain Coupling این مفهوم به میزان وابستگی و ارتباط متقابل بین بخش‌ها یا دامنه‌های مختلف یک سیستم نرم‌افزاری اشاره دارد. به زبان ساده، وقتی دو بخش از سیستم شدیداً به هم متصل باشند (Coupling بالا)، تغییر در یکی باعث خرابی یا نیاز به تغییر در دیگری می‌شود. در معماری‌های نوین (مثل میکروسرویس‌ها)، هدف این است که این وابستگی را به حداقل برسانند تا هر بخش بتواند به صورت مستقل توسعه یافته و تغییر کند.

ب

Data Decomposition تجزیه داده‌ها به معنای شکستن و تقسیم کردن یک پایگاه داده بزرگ و یکپارچه به دیتابیس‌های کوچک‌تر و مجزا است؛ به طوری که هر میکروسرویس یا دامنه فقط به داده‌های مربوط به خودش دسترسی داشته باشد. به زبان ساده، یعنی به جای اینکه همه بخش‌های یک سیستم اطلاعات خود را در یک انبار مشترک بریزند، هر بخش انبار اختصاصی کوچک خودش را داشته باشد تا تداخلی در کار هم ایجاد نکنند.

ج

Event-Driven Communication Pattern یک روش ارتباطی بین سرویس‌های مختلف است که در آن، اجزای سیستم از طریق ارسال و دریافت رویدادها با یکدیگر صحبت می‌کنند. به زبان ساده، وقتی در یک بخش از سیستم اتفاقی می‌افتد (مثلاً خرید کاربر ثبت شد)، آن بخش یک اعلان یا سیگنال به هوا می‌فرستد. سرویس‌های دیگر که به این موضوع علاقه دارند، این سیگنال را دریافت کرده و کار خودشان را انجام می‌دهند، بدون اینکه نیاز باشد مستقیماً با سرویس اولیه تماس بگیرند یا منتظر پاسخ آن بمانند.

پاسخ مسئله‌ی ۲.

مشکل اصلی در این سناریو، تجزیه زودرس سیستم یا همان Premature Decomposition (یا اصطلاحاً Distributed Monolith) است. تیم توسعه پیش از درک کامل بیزینس و مرزهای دامنه مسئله (Bounded Contexts)، سیستم را به میکروسرویس‌های متعدد تقسیم کرده است. این کار بدون وجود نیاز واقعی به مقیاس‌پذیری، تنها پیچیدگی‌های عملیاتی را به شدت افزایش داده است.

شروع مستقیم با معماری میکروسرویس در ابتدای پروژه به دلایل زیر اشتباه است:

- عدم شناخت دامنه: وقتی مرزهای بیزینس شفاف نیست، میکروسرویس‌ها به اشتباه تفکیک می‌شوند. این امر منجر به ایجاد Domain Coupling شدید شده و تغییر در یک ویژگی، نیازمند تغییر همزمان در چندین سرویس و API می‌شود.
- ایجاد منولیت توزیع‌شده: سیستم تمام معایب منولیت (وابستگی شدید) و تمام معایب میکروسرویس (پیچیدگی شبکه، دشواری تست و باگ‌های مرزی) را یکجا دارد.
- کاهش سرعت توسعه: نوشتن تست‌های End-to-End در یک سیستم توزیع‌شده با دیتابیس‌های مجزا بسیار سخت‌تر از یک سیستم یکپارچه است که نتیجه آن افت شدید سرعت تحویل Feature هاست.

رویکرد استاندارد و پیشنهادی، استراتژی Monolith First است:

۱. شروع با یک منولیت ماژولار: سیستم باید در ابتدا به صورت یکپارچه اما با کدهای ماژولار و ساختاریافته توسعه یابد. در این حالت، مرزها در سطح کد (پکیج‌ها یا کامپوننت‌ها) تعریف می‌شوند و ارتباطات به جای شبکه، از طریق فراخوانی توابع داخل حافظه است.
۲. کشف مرزهای دامنه: با رشد پروژه و شناخت دقیق‌تر رفتار کاربران و بیزینس، مرزهای واقعی دامنه‌ها (Subdomains) مشخص می‌شوند.
۳. مهاجرت تدریجی در صورت نیاز: هر زمان که یک ماژول خاص از نظر فنی نیاز به مقیاس‌پذیری مستقل پیدا کرد یا تیم‌های توسعه آن‌قدر بزرگ شدند که روی یک کدبیس تداخل داشتند، آن ماژول به صورت یک میکروسرویس مجزا جدا می‌شود.

پاسخ مسئله‌ی ۳.

تفاوت دو رویکرد بر اساس معیارهای مطرح شده:

۱. Coupling:

- همگام: دارای Coupling بالایی است. سرویس سفارش باید مستقیماً آدرس (IP/URL) و ساختار سرویس‌های پرداخت و ایمیل را بشناسد.
- ناهمگام: دارای Loose Coupling است. سرویس سفارش فقط رویداد را به Message Broker می‌فرستد و هیچ شناختی از گیرنده‌ها ندارد.

۲. Latency:

- همگام: تاخیر کل برابر است با مجموع زمان پاسخ‌دهی تمام سرویس‌ها. سرویس سفارش باید منتظر بماند تا پرداخت و ایمیل کارشان تمام شود که سرعت را پایین می‌آورد.
- ناهمگام: تاخیر برای کاربر به شدت کاهش می‌یابد؛ زیرا به محض ثبت سفارش و ارسال ایونت، پاسخ به کاربر برگشت داده می‌شود و بقیه کارها در پس‌زمینه انجام می‌شود.

۳. Fault Tolerance:

- همگام: پایین است. اگر سرویس ایمیل یا پرداخت در آن لحظه در دسترس نباشد، کل فرآیند ثبت سفارش با خطا مواجه می‌شود.
- ناهمگام: بالا است. اگر سرویس ایمیل خراب باشد، پیام در Queue یا Broker باقی می‌ماند و به محض بالا آمدن سرویس، پردازش می‌شود؛ بدون اینکه تراکنش اصلی خراب شود.

۴. Scalability:

- همگام: به سختی مقیاس می‌شود؛ زیرا فشار کاری مستقیماً به سرویس‌های بعدی منتقل می‌شود و در صورت هجوم کاربران، گلوگاه ایجاد می‌شود.
- ناهمگام: بسیار بالا است. Message Broker مانند یک ضربه‌گیر عمل کرده و پیام‌ها را ذخیره می‌کند؛ سرویس‌ها می‌توانند متناسب با توان خود و به صورت ناهمزمان پیام‌ها را مصرف کنند.

با توجه به تحلیل بالا، بهترین استراتژی ترکیب هر دو رویکرد به شکل زیر است:

۱. ارتباط با سرویس پرداخت با رویکرد همگام:

فرآیند پرداخت یک عملیات حیاتی و وابسته به زمان است. سرویس سفارش باید فوراً متوجه شود که آیا پرداخت با موفقیت انجام شد یا خیر تا وضعیت سفارش را مشخص کند. بنابراین استفاده از یک فراخوانی همگام مانند REST در اینجا منطقی‌تر است.

۲. ارتباط با سرویس ارسال ایمیل با رویکرد ناهمگام:

ارسال ایمیل تأییدیه خرید، تأثیری روی موفقیت آمیز بودن خود فرآیند ثبت سفارش ندارد. کاربر نباید معطل ارسال ایمیل بماند. بنابراین، سرویس سفارش پس از نهایی شدن پرداخت، رویدادی مانند OrderConfirmed را در Message Broker منتشر می‌کند و سرویس ایمیل آن را به صورت ناهمگام برمی‌دارد و پردازش می‌کند.

پاسخ مسئله‌ی ۴.

برای این پلتفرم، سیستم را به دامنه‌های مجزا با مالکیت داده اختصاصی (Database-per-Service) تقسیم می‌کنیم:

- سرویس احراز هویت و کاربران (Auth & User Service): مدیریت ثبت‌نام، پروفایل دانشجویان/مدرسان و نقش‌ها.
- سرویس کاتالوگ دوره‌ها (Course Catalog Service): مشاهده دوره‌ها، جستجو و بارگذاری محتوا (ویدئوها و متون) توسط مدرسان.
- سرویس سفارش و پرداخت (Order & Payment Service): مدیریت سبد خرید، صدور فاکتور و اتصال به درگاه پرداخت.
- سرویس تعاملی و تمرین‌ها (Progress & Assignment Service): ثبت وضعیت پیشرفت دانشجویان، ارسال تمرین‌ها و فرآیند تصحیح.
- سرویس گواهی‌نامه (Certificate Service): صدور و اعتبارسنجی گواهی پایان دوره به صورت خودکار.
- سرویس پیشنهادات و رتبه‌بندی (Recommendation & Review Service): ثبت امتیازها و سیستم پیشنهاد دوره بر اساس رفتارهای قبلی کاربر.
- سرویس اعلان‌ها (Notification Service): ارسال ایمیل، پیامک و اعلان‌های درون‌برنامه‌ای در زمان کمپین‌ها یا انتشار دوره جدید.

جهت حفظ پایداری و کارایی سیستم در زمان اوج ترافیک، الگوهای ارتباطی زیر پیاده‌سازی می‌شوند:

۱. ارتباط همگام (Synchronous - HTTP/gRPC):

- کاربرد: در لایه API Gateway برای هدایت درخواست‌های مستقیم کاربر و همچنین در مواقعی که نیاز به پاسخ آنی است؛ مانند تایید نهایی تراکنش توسط Payment Service که Order Service باید فوراً از آن مطلع شود.

۲. ارتباط ناهمگام (Asynchronous - Event-Driven via Message Broker):

- خرید موفق: پس از پرداخت، Payment Service رویداد OrderPaid را منتشر می‌کند. سرویس‌های تعاملی (برای باز شدن دسترسی دوره)، گواهی‌نامه (برای بررسی وضعیت نهایی بعداً) و اعلان‌ها این رویداد را مصرف می‌کنند.
- انتشار دوره جدید یا کمپین: سرویس کاتالوگ رویداد CoursePublished را منتشر کرده و Notification Service به صورت ناهمگام شروع به ارسال ایمیل‌های گروهی بدون درگیر کردن منابع اصلی وب‌سایت می‌کند.

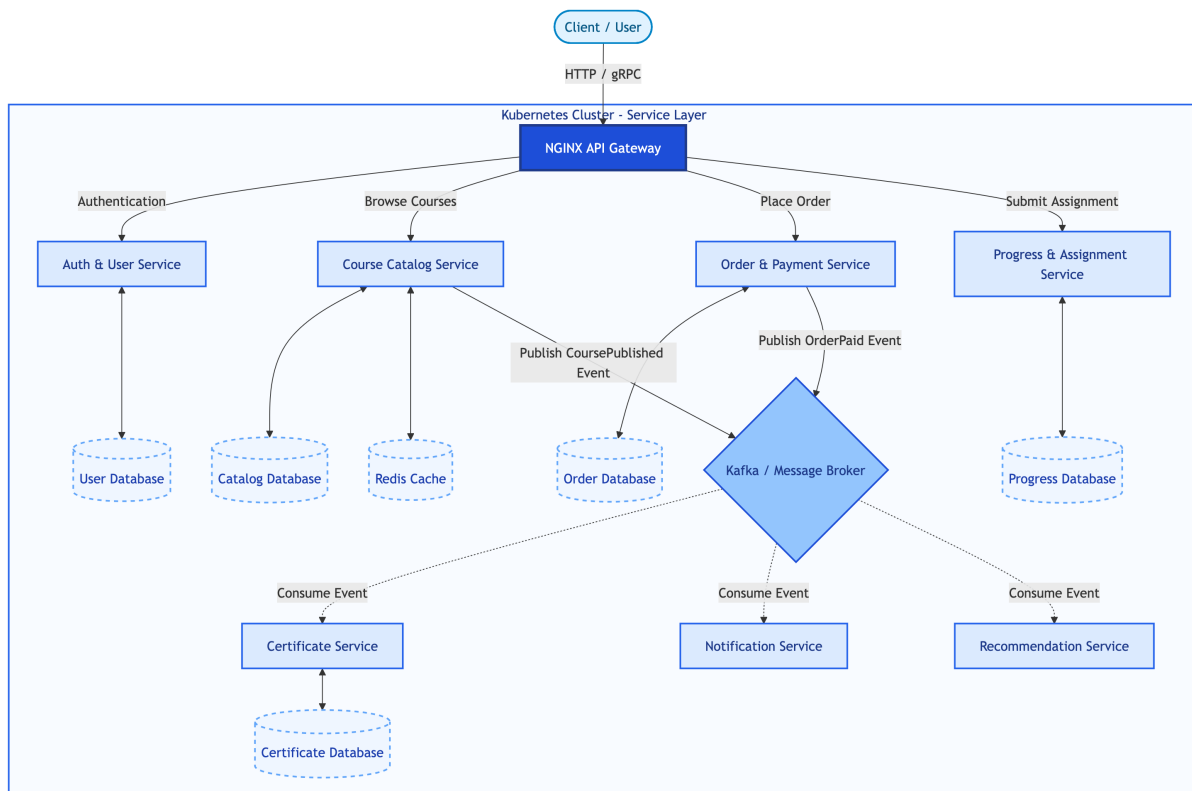
با توجه به مجزا بودن دیتابیس‌ها، امکان استفاده از ACID سنتی وجود ندارد. برای فرآیندهای چندمرحله‌ای مثل «خرید دوره ← کسر موجودی/تایید درگاه ← فعال‌سازی دسترسی دوره»، از الگوی ساگا (Saga Pattern) مبتنی بر ارکستراسیون استفاده می‌شود:

- تراکنش‌های جبرانی: اگر در مرحله آخر (مثلاً فعال‌سازی دوره) خطایی رخ دهد، رویداد جبرانی (Rollback Event) صادر می‌شود تا تراکنش مالی در سرویس پرداخت برگشت داده شود (Refund) و سیستم به وضعیت پایدار قبلی برسد.

با توجه به اینکه در زمان کمپین‌ها تعداد کاربران به طور ناگهانی افزایش می‌یابد، تمهیدات زیر الزامی است:

- Auto-scaling در سطح کانیتینرها: استفاده از کلاستر Kubernetes و تنظیم HPA بر اساس میزان مصرف CPU و نرخ درخواست‌ها، تا سرویس‌های پرمصرف مثل کاتالوگ و احراز هویت به سرعت تکثیر شوند.

- لایه کشینگ توزیع‌شده: استفاده از Redis برای کش کردن اطلاعات سنگین و پربازدید (مانند ساختار دوره‌ها و ویدئوها) تا فشار روی پایگاه داده‌های اصلی کاهش یابد.
- کنترل نرخ درخواست (Rate Limiting): پیاده‌سازی الگوریتم‌های Rate Limiting در سطح API Gateway برای جلوگیری از حملات DDoS و مسدود کردن درخواست‌های مخرب یا اسپم در زمان کمپین‌ها.
- ضربه‌گیری با Message Broker: هدایت حجم عظیمی از کارهای غیرضروری (مثل ثبت لاگ‌های پیشرفت ویدئو یا صف‌های ارسال اعلان) به ابزارهایی مانند Kafka یا RabbitMQ جهت تعدیل بار.



پاسخ مسئله‌ی ۵.

آ: nginx

کاربرد: به عنوان API Gateway، لود بالانسر و Reverse Proxy عمل می‌کند. درخواست‌های ورودی کاربران را دریافت کرده و به میکروسرویس مربوطه هدایت می‌کند. همچنین وظایفی مثل SSL Termination و کش کردن محتوای استاتیک را انجام می‌دهد.

مثال واقعی: روی یک سرور، فرانت‌اند روی app.example.com و بک‌اند روی api.example.com قرار دارند؛ nginx بر اساس ساب‌دامین، درخواست‌ها را به‌ترتیب به سرویس فرانت (مثلا React روی پورت ۳۰۰۰) و سرویس API (مثلا روی پورت ۸۰۰۰) هدایت می‌کند.

ب: Prometheus

کاربرد: ابزاری برای مانیتورینگ و جمع‌آوری متریک‌ها به صورت Pull-based است. وضعیت سلامتی، میزان مصرف CPU/RAM و تعداد درخواست‌های هر میکروسرویس را ثبت می‌کند.

مثال واقعی: سرویس پرداخت متریکی به نام successful_payments_total را اکسپوز می‌کند و Prometheus هر ۳۰ ثانیه آن را می‌خواند تا در صورت افت ناگهانی تراکنش‌ها، هشدار صادر کند.

پ: RabbitMQ

کاربرد: یک Message Broker مبتنی بر پروتکل AMQP است که برای ارتباطات ناهمگام و مطمئن بین سرویس‌ها استفاده می‌شود. پیام‌ها را در صف‌ها نگه می‌دارد تا سرویس مقصد آن‌ها را پردازش کند.

مثال واقعی: در یک سیستم فروشگاهی، پس از خرید موفق، سرویس سفارش پیامی به صف shipping-queue می‌فرستد تا سرویس ارسال کالا بعداً آن را بردارد و پردازش کند.

ت: Celery

کاربرد: یک صف وظایف توزیع‌شده برای پایتون است که کارهای سنگین، زمان‌بر یا دوره‌ای را خارج از چرخه اصلی درخواست کاربر اجرا می‌کند.

مثال واقعی: وقتی کاربر درخواست خروجی اکسل از گزارش‌های یک‌ساله خود را می‌دهد، سرویس وب این کار سنگین را به Celery می‌سپارد تا در پس‌زمینه انجام شود و به کاربر پیام گزارش در حال آماده‌سازی است را فوراً نشان می‌دهد.

ث: ELK (Elasticsearch, Logstash, Kibana)

کاربرد: پشته‌ای برای مدیریت، جمع‌آوری و تحلیل لوکال و متمرکز لاگ‌ها در سیستم‌های توزیع‌شده است. Logstash لاگ‌ها را جمع‌آوری و فیلتر می‌کند، Elasticsearch آن‌ها را ایندکس و ذخیره می‌کند و Kibana محیط گرافیکی نمایش آن‌هاست.

مثال واقعی: وقتی یک خطای ناشناخته در فرآیند ثبت‌نام رخ می‌دهد، برنامه‌نویس به جای بررسی لاگ‌های تک‌تک سرورها، در Kibana شناسه کاربر را جستجو می‌کند تا تمام لاگ‌های مربوط به او را در میکروسرویس‌های مختلف ببیند.

ج: Redis

کاربرد: یک پایگاه داده درون حافظه‌ای بسیار سریع است که بیشتر به عنوان لایه کش، مدیریت نشست‌ها یا Rate Limiter استفاده می‌شود.

مثال واقعی: اطلاعات پروفایل کاربران یا کاتالوگ محصولات پربازدید در Redis کش می‌شود تا نیازی به کوئری زدن مداوم به دیتابیس‌های اصلی و کندتر نباشد.

ج: Kubernetes

کاربرد: پلتفرم ارکستراسیون کانتینرها (Container Orchestration) است. وظیفه مدیریت چرخه حیات، استقرار اتوماتیک، مقیاس‌پذیری خودکار (Auto-scaling) و حفظ پایداری کانتینرهای میکروسرویس‌ها را بر عهده دارد.

مثال واقعی: در زمان کمپین‌های تخفیفی که ترافیک سایت ۱۰ برابر می‌شود، Kubernetes به طور خودکار تعداد Podهای سرویس کاتالوگ را افزایش می‌دهد تا سیستم زیر بار ترافیک ن خوابد.

ح: Kafka

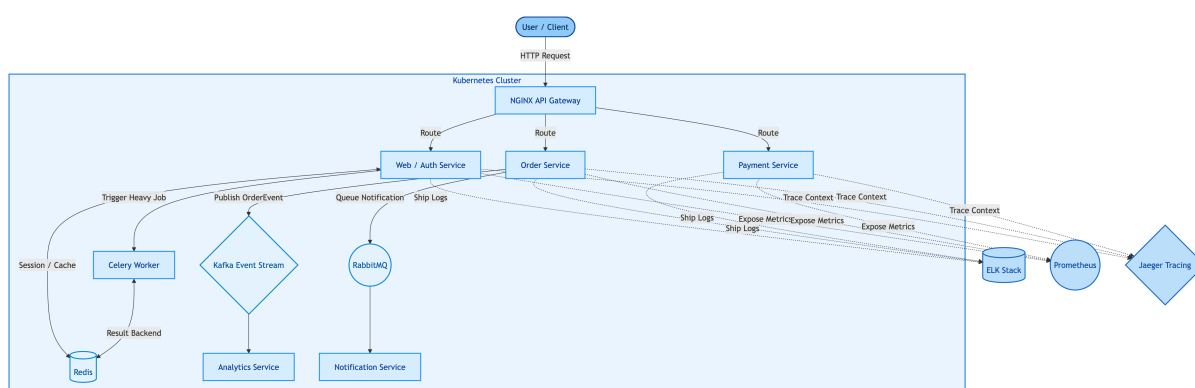
کاربرد: یک پلتفرم توزیع‌شده برای پردازش و انتقال جریان داده‌ها (Event Streaming) با حجم و سرعت بسیار بالا است. بر خلاف واسطه‌های سنتی، پیام‌ها را به صورت پایدار روی دیسک ذخیره می‌کند و چندین مصرف‌کننده می‌توانند تاریخچه داده‌ها را بازخوانی کنند.

مثال واقعی: در سیستم‌های تاکسی اینترنتی، جریان مختصات جغرافیایی (GPS) راننده‌ها به صورت مداوم به Kafka فرستاده می‌شود تا سرویس‌های تخمین قیمت، نقشه و بررسی رفتار راننده همزمان از آن استفاده کنند.

خ: Jaeger

کاربرد: ابزاری برای ردیابی توزیع‌شده (Distributed Tracing) است. مسیر حرکت یک درخواست کاربر را در میان میکروسرویس‌های مختلف دنبال کرده و زمان صرف شده در هر مرحله را به صورت گرافیکی نمایش می‌دهد تا گلوگاه‌ها کشف شوند.

مثال واقعی: اگر لود شدن صفحه سبد خرید ۵ ثانیه طول بکشد، با استفاده از Jaeger مشخص می‌شود که مثلاً ۲.۴ ثانیه از این زمان مربوط به کوئری دیتابیس در سرویس تخفیف بوده است.



پاسخ مسئله‌ی ۶.

در این بخش، یک API مبتنی بر GraphQL برای سیستم مدیریت کارهای روزانه طراحی و پیاده‌سازی شده است. کاربران می‌توانند کار (Task) بسازند، ویرایش یا حذف کنند، وضعیت را میان TODO، DOING و DONE جابه‌جا کنند و کارها را با برچسب (Tag) سازمان‌دهی کنند.

استک فناوری و توجیه انتخاب

- Node.js و TypeScript: زبان و محیط اجرای پروژه؛ با توجه به اکوسیستم قوی GraphQL در Node انتخاب شد.
- NestJS: فریم‌ورک ماژولار سمت سرور که تزریق وابستگی، ساختار لایه‌ای و یکپارچگی رسمی با GraphQL را فراهم می‌کند.
- Apollo Server: موتور اجرای GraphQL؛ محیط تعاملی Apollo Sandbox روی مسیر graphql/ برای کاوش اسکیم و تست عملیات در دسترس است.
- @nestjs/graphql با رویکرد Code-first: اسکیم از روی کلاس‌ها و decoratorهای TypeScript تولید می‌شود و فایل src/schema.gql به صورت خودکار ساخته می‌شود؛ منبع حقیقت یکی است و type-safety حفظ می‌شود.
- SQLite + Prisma: لایه دسترسی به داده با migration و کلاینت تایپ‌شده؛ SQLite پایگاه سبک و مناسب تمرین است و پشت پورت ریپازیتوری قابل تعویض با PostgreSQL است.
- class-transformer / class-validator: اعتبارسنجی ورودی‌های Input قبل از رسیدن به لایه دامنه.
- DataLoader: حل مسئله کلاسیک N+1 برای فیلد رابطه‌ای Tag.tasks.
- graphql-depth-limit: محدود کردن عمق کوئری‌های تو در تو (حداکثر ۶) برای جلوگیری از سوءاستفاده.
- pino / nestjs-pino: لاگ ساختاریافته با شناسه همبستگی (request id) و زمان اجرای هر عملیات GraphQL.
- Jest + supertest: تست واحد (بدون دیتابیس) و تست e2e روی اپ واقعی.
- Docker / docker-compose و GitHub Actions: استقرار و اجرای خودکار CI (لینت، بیلد، یونیت و e2e).

معماری

پروژه از Clean Architecture پیروی می‌کند: وابستگی‌ها فقط به سمت داخل هستند و لایه دامنه هیچ وابستگی به Prisma، NestJS یا GraphQL ندارد. هر ویژگی (tasks و tags) به چهار لایه تقسیم شده است:

۱. domain: موجودیت‌ها، اینورینت‌ها، enumها، پورت ریپازیتوری (اینترفیس) و خطاهای دامنه‌ای.
۲. application: یک Use Case به‌ازای هر عملیات (مثلاً CreateTask، ListTasks، UpdateTag).
۳. infrastructure: آداپتر Prisma و mapper تبدیل ردیف دیتابیس ↔ موجودیت دامنه.
۴. presentation: Object Typeها، Inputها، Resolverها و در صورت نیاز DataLoader.

مسیر یک درخواست به صورت خلاصه چنین است:

۱. دریافت درخواست POST /graphql
۲. اعتبارسنجی اسکیم و عمق کوئری

۳. اعتبارسنجی ورودی با GqlValidationPipe

۴. اجرای Resolver

۵. فراخوانی Use Case

۶. اعمال قوانین در موجودیت دامنه

۷. ذخیره/خواندن از طریق ریپازیتوری Prisma

۸. دسترسی به SQLite

نکته طراحی مهم: فیلد Tag.tasks داخل مازول tasks (TagTasksResolver) پیاده شده تا وابستگی مازول‌ها یک‌طرفه بماند (tasks به tags) و چرخه وابستگی ایجاد نشود.

ساختار پوشه‌ها

```

1 src/
2   common/           # config, errors, filters, pipes, pagination
3   infrastructure/prisma/ # PrismaService
4   health/           # REST GET /health
5   modules/
6     tasks/{domain,application,infrastructure,presentation}
7     tags/{domain,application,infrastructure,presentation}
8     schema.gql       # auto-generated GraphQL schema
9 prisma/schema.prisma
10 postman/NestJS-GraphQL.postman_collection.json
11 test/               # e2e

```

مدل داده

دو موجودیت اصلی با رابطه چندبه‌چند وجود دارد:

- Task: عنوان اجباری، توضیحات اختیاری، وضعیت، اولویت، تاریخ سررسید، برچسب‌ها، createdAt و updatedAt.
- Tag: نام یکتا، رنگ هگزادسیمال اختیاری، لیست کارهای مرتبط.

چون SQLite در Prisma از enum بومی پشتیبانی نمی‌کند، وضعیت به‌صورت رشته و اولویت به‌صورت عدد صحیح (1=LOW تا 3=HIGH) ذخیره می‌شود تا سورت اولویت معنادار باشد؛ ترجمه به enum دامنه در mapper انجام می‌شود.

کوئری‌ها

- tasks(filter, pagination, sort) — فیلتر بر اساس وضعیت، اولویت، برچسب، جستجو در عنوان/توضیح و بازه سررسید؛ صفحه‌بندی skip/take (حداکثر take=50)؛ مرتب‌سازی روی چند فیلد.
- task(id) — در صورت نبودن، null برمی‌گرداند.
- tags و tag(id).

میوتیشن‌ها

- changeTaskStatus / deleteTask / updateTask / createTask
- deleteTag / updateTag / createTag

نکات پیاده‌سازی مهم

- DataLoader برای Tag.tasks: جلوگیری از N+1؛ تمام tagIdهای یک درخواست در یک کوئری دسته‌ای واکشی می‌شوند.
- اعتبارسنجی ورودی: محدودیت طول عنوان، فرمت رنگ، سقف صفحه‌بندی و غیره در لایه presentation؛ قوانین کسب‌وکار (مثل عنوان خالی) در موجودیت دامنه.
- سلامت سرویس: مسیر REST با آدرس /health برای Docker healthcheck.
- تست: بیش از ۶۰ تست واحد روی دامنه و use case ها با ریپازیتوری درون‌حافظه‌ای، و بیش از ۲۰ تست e2e روی دیتابیس موقت.

اجرا و نحوه تست

```
1 cp .env.example .env
2 npx prisma migrate dev
3 npm run start:dev
4 # open http://localhost:3000/graphql (Apollo Sandbox)
```

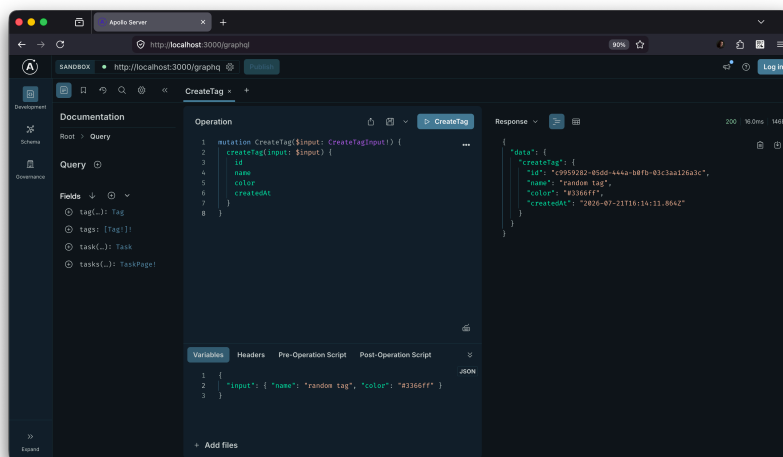
برای پوشش کامل عملیات (ساخت/ویرایش/حذف کار و برچسب، تغییر وضعیت، فیلتر، سورت، صفحه‌بندی و نمونه‌های خطا) می‌توان از یکی از این دو روش استفاده کرد:

- Apollo Sandbox: اسکیم را در سمت چپ مرور کرده و کوئری/میوتیشن را با autocomplete اجرا کنید.
- کالکشن Postman: فایل postman/NestJS-GraphQL.postman_collection.json را ایمپورت کنید و پوشه‌ها را به ترتیب Health، Tags، Tasks و Error examples اجرا کنید. شناسه‌های ساخته‌شده در متغیرهای کالکشن ذخیره می‌شوند و درخواست‌های بعدی به‌صورت زنجیره‌ای کار می‌کنند.

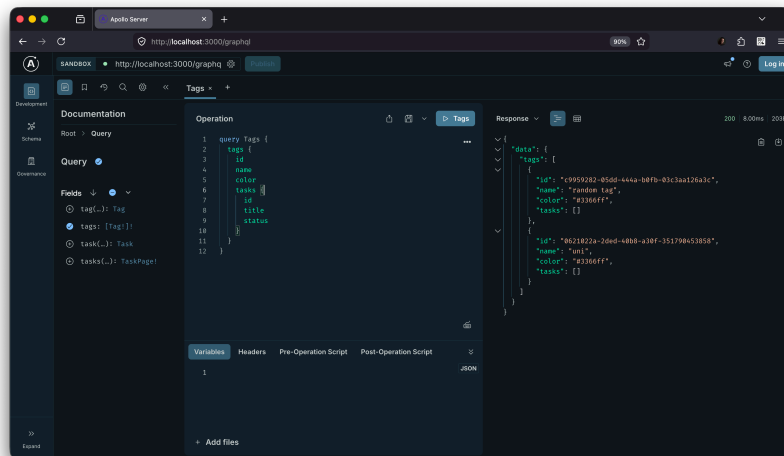
نمونه‌های اجراشده

در ادامه سه نمونه از عملیات اصلی برچسب‌ها که در Apollo Sandbox اجرا شده‌اند آورده شده است. باقی درخواست‌ها (از جمله CRUD کامل کارها، فیلتر، سورت و نمونه‌های خطا) از طریق همان Sandbox یا کالکشن Postman قابل تست هستند.

میوتیشن createTag با نام random tag و رنگ #3366ff اجرا شد و شناسه UUID همراه با createdAt برگردانده شد.



کوئری tags فیلدهای id، name، color و رابطه تو در توی تسک‌ها (tasks با فیلدهای id، title و status) را درخواست می‌کند. این سناریو همان مسیری است که DataLoader برای جلوگیری از N+1 به کار می‌آید.



میوتیشن updateTag نام را به new random tag و رنگ را به #00aa55 تغییر داد و آبجکت به‌روزشده را برگرداند.

